

Chapter 1

The Essential of Software Requirement

© Karl E. Wieggers



Topics

1.1 Introduction

1.2 Difficulty of defining Requirements

1.3 Definition of Requirement

1.4 Types and levels of Requirement

1.5 Requirement Engineering

1.6 Critical Role of Requirement

(Bad and Good Requirement)



1.1 Introduction

Failure to understand
and
control requirements
is
the number 1 cause of project failure.



“The single hardest part of building a software system is deciding precisely what to build. ... No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.”

Frederick Brooks in *No Silver Bullet: Essence and Accidents of Software Engineering*



➤ The Customer's Frustration

- Using a product that doesn't support an essential task.
- Being at the mercy of a developer whose priorities may not be yours

➤ The Developer's Frustration

- Learning of essential functions after system has been implemented
- Being requested to modify a feature that is exactly what was asked for



1.2 Why is Difficult?

➤ It involves the largest number of stakeholders

- Customers
- Users
- Requirements analysts
- Developers
- Testers
- Document writers
- Managers
- Legal staff
- Manufacturing staff
- Support staff (sales, help desk,...)



➤ Shortcomings of Requirements Gathering

- Informal gathering process
- Implied functionality
- Wrong or unspoken assumptions
- Inadequate (incomplete, ambiguous, conflicting) requirement specifications
- Casual change process— decisions not recorded, undesirable changes creep in



1.3 Definition of Requirement

➤ Informal Definitions

- External behavior and appearance of system
- “What should be implemented.”
- “System property or attribute, perhaps a constraint.”



More Formal Definitions (IEEE Glossary)

- “Condition or capability needed by a user to solve a problem or achieve an objective”
- “Condition or capability that must be met by a system...to satisfy a contract...”
- “A documented representation of a condition or capability in part 1 or 2”



Requirements are a specification that should be implemented.

They are descriptions of how the system should behave , or a systems property or attribute.

They may be a constraint to the development process of the system

(Summerville)



1.4 Types and Levels of Requirement

➤ Functional Requirement

Description of a behavior that the system will show under a specific condition.

It is influenced by:

- Business Requirements (vision and scope)
- User Requirements (use cases)
- System Requirements (when there are multiple subsystems)

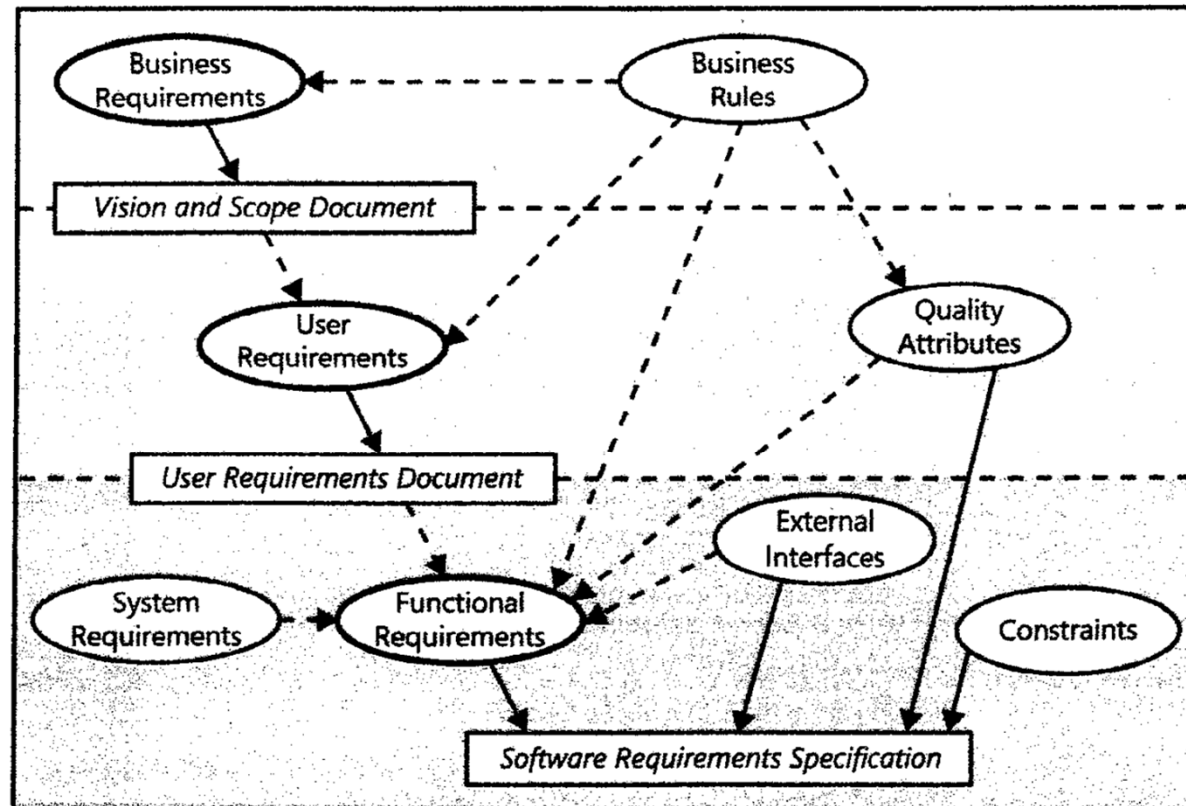


FIGURE 1-1 Relationships among several types of requirements information. Solid arrows mean "are stored in"; dotted arrows mean "are the origin of" or "influence."

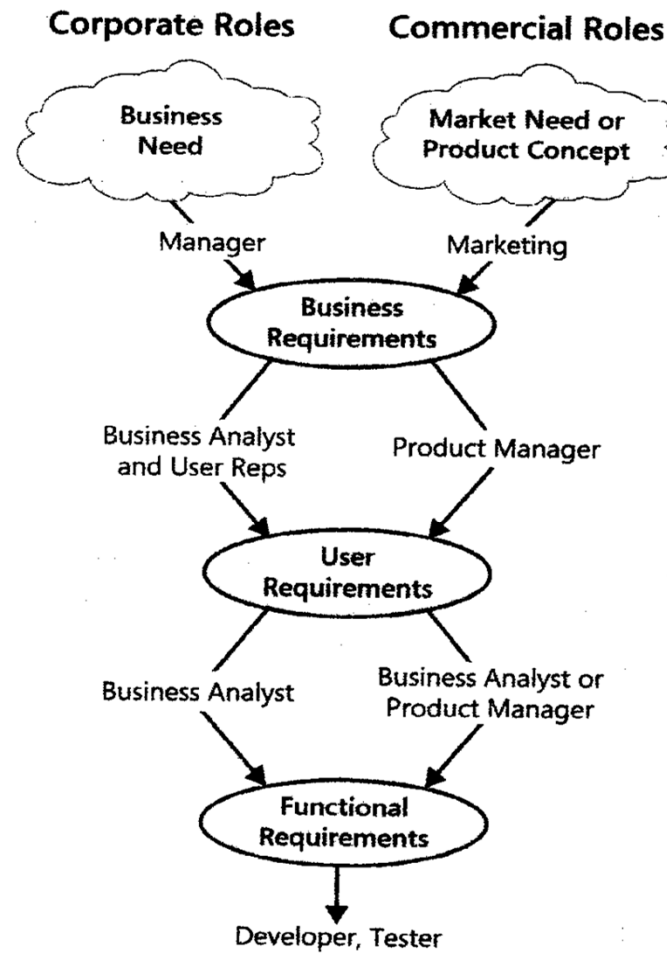


FIGURE 1-3 An example of how different stakeholders participate in requirements development.



➤ Nonfunctional Requirement

Description of a property or characteristic that a system must exhibit or restriction/constraint it must respect.

- Quality Attributes (performance, availability, security, usability, integrity, efficiency, robustness)
- Constraints (limits on choices)



➤ What's Excluded?

- No design or implementation detail
 - It's too early, the "what" isn't finished
 - It's too limiting, designers need freedom
- No project planning (it's not the right place)
- No testing information (but testers will use requirements to determine test goals)

1.5 Requirement Engineering (Development and Management)

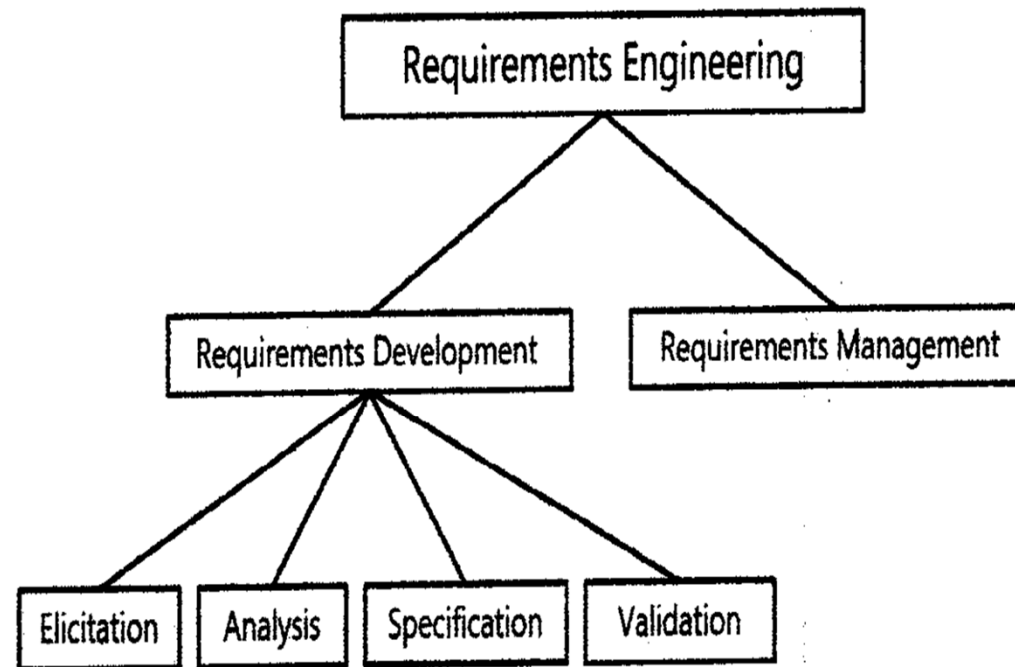


FIGURE 1-4 Subdisciplines of software requirements engineering.



1.5.1 Development involves 4 activities

➤ Elicitation

- Identifying user classes
- Bringing out needs from representative users
- Understanding their tasks and goals
- Understanding the related business objectives



➤ Analysis

- Distinguish task goals from functional and nonfunctional requirements, rules, suggestions, extraneous information
- Allocate top-level requirements to architecture components
- Understand importance of quality attributes
- Negotiate priorities



➤ Specification

- Translate user needs into written specifications and models

➤ Validation

- Review documented requirements
 - Ensure common understanding between user and developer
 - Correct problems before development starts



1.5.2 Requirements Management

It involves:

- “Establishing and maintaining an agreement with the customer...”
- Follows the Requirements Development activity
- Maintain a controlled baseline



Some Activities are:

- Defining baseline— snapshot of current agreement
- Reviewing change proposals, evaluating impact before approval
- Incorporating changes in controlled way
- Keeping project plan current
- Negotiating new commitments (cost, schedule)
- Tracing individual requirements to design, code, test
 - Ensure that everything needed is included
 - Prevent unwanted features from appearing
- Tracking status and change activity
 - Know what's proposed, in-work, finished
 - Know the trends

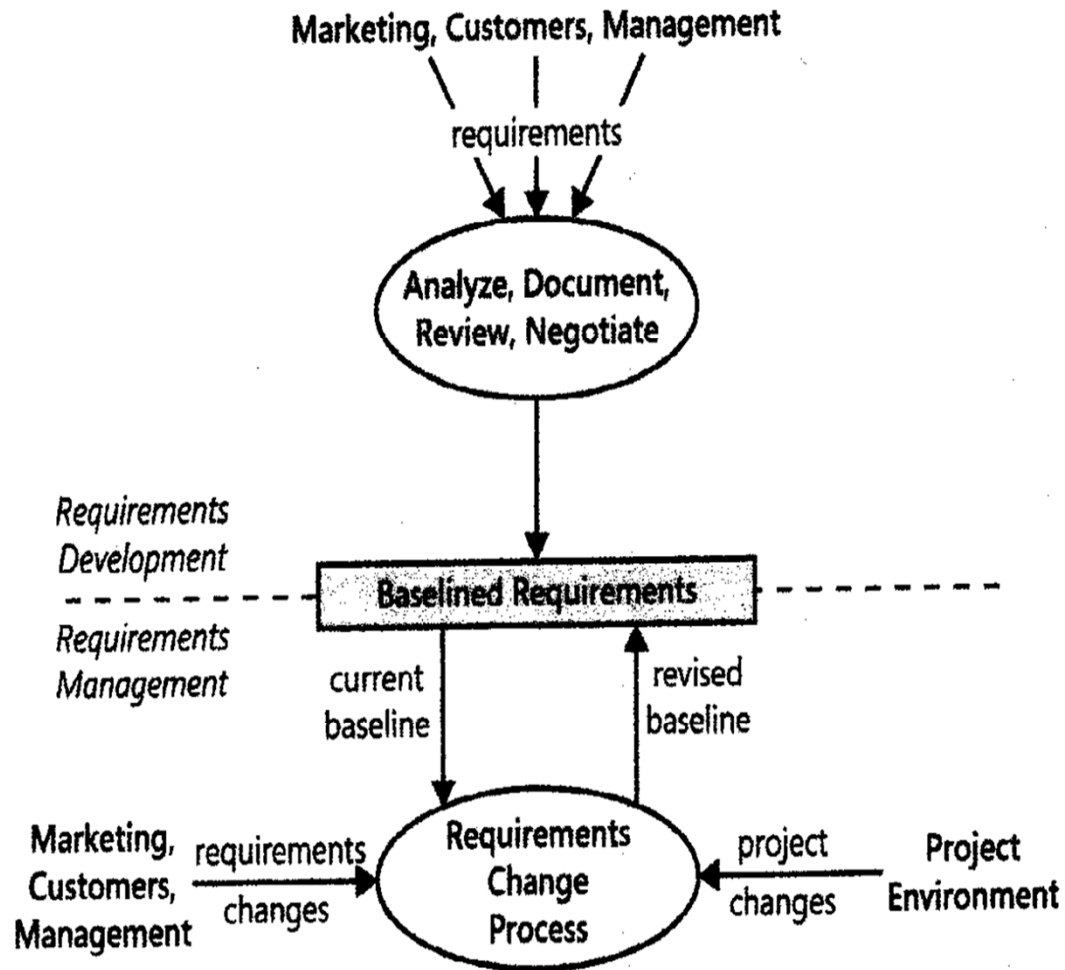


FIGURE 1-5 The boundary between requirements development and requirements management.



1.6 Critical Role of Requirement

- There Are Always Requirements
- Even for internal projects, there's still a user group with needs and goals



➤ Consequences of Bad Requirements

- Rework— doing the design, the code, the testing over (and over, and over, and ...)
- Cost of rework grows as discovery time becomes later



➤ Causes and Effects of Bad Requirements

- Insufficient user involvement
 - leads to late-breaking requirements
 - delays project completion
- Creeping user requirements
 - frequent changes too easily accepted by developers
 - no adherence to vision and scope
 - architecture stability decays



- Ambiguous requirements
 - Multiple interpretations
 - Different expectations by stakeholders
 - Developers waste time filling in details
- Gold-plating
 - Developers add features to impress customer
 - May add little value, but still cost time and money, detracting from actual needs



- Minimal specifications
 - Saving writing time costs design and implementation time
- Overlooked users
 - Unknown stakeholders aren't heard until very late, when they say the system's not usable
- Inaccurate planning
 - Poorly understood requirements lead to optimistic over-commitment



➤ Benefits of Good Requirements

- Fewer defects
- Less rework
- Faster development
- Better estimates
- Less chaos
- Higher satisfaction for user and developer



➤ Excellent Requirements

- Complete
 - No missing details; use TBD to flag gaps in knowledge
- Correct
 - From a credible user, preferably several
- Feasible
 - Avoid impossible (and extremely difficult) requirements
 - Use incremental development, prototypes to evaluate questionable requirements

- 
- **Necessary**
 - Make sure user has authority
 - **Prioritized**
 - Prepare for trade-offs, problems later in project
 - **Unambiguous**
 - Ensure single, consistent interpretation by all readers
 - Define specialized terms, avoid software jargon
 - **Verifiable**
 - Correctness of un-testable requirements depends on opinion, not objective evidence



➤ Excellent Specification

- Complete
 - No missing features
 - Focused on user tasks, not system functions
- Consistent
 - No conflicts between functional or nonfunctional requirements
- Originator recorded, to enable resolution when needed
- Modifiable
 - Revised with change history
 - Each requirement uniquely identified
- Traceable



END